Lecture 1: Course Introduction

& Overview

Course Orientation • Intro to Distributed Systems • Real-world Examples

Course: Distributed Systems

Part 1: Course Orientation

Objectives, Assessments, and Policies

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 1: Course Introduction and Overview	Understand the fundamental concepts and real-world applications of distributed systems.	Introduces the definition of distributed systems, course objectives, and real-world examples (Google, Facebook, Cloud Computing). It explores why modern applications use multiple computers instead of one powerful machine.

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 2: Basic Concepts of Distributed Computing	Identify the components and key design principles of distributed systems.	Covers the definition of distributed computing and its main components: nodes, processes, communication links, and resource sharing. Introduces key concepts like transparency, Availability, and scalability.

Transparency → Can it hide its complexity from users?
Scalability → Can the system grow with more users?
Consistency → All servers show the same data.
Fault Tolerance → Can it keep working when failures occur?
Availability → The system always responds.

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture3: socket	How the nodes can be communicated with each other .	Introduce the basic concepts and simple programs to understand how to create a client-server socket .

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lectures 4: multi-threads	how the multitask in application can be executed at the same time.	Introduce the basic concepts and simple programs to understand how to create- execute threads

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 5: Goals of Distributed Systems	the primary goals and design trade-offs in building distributed systems.	Details the main goals: making resources accessible, ensuring distribution transparency (access, location, failure, etc.), maintaining openness, and achieving scalability. Discusses the trade-offs between them.

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 6: Characteristics and Design Issues	Recognize the characteristics and fundamental design challenges of distributed systems.	Explains core characteristics (concurrent execution, lack of a global clock, independent failures) and major design challenges including heterogeneity, security, scalability, fault tolerance, and quality of service.

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 7: Types of Distributed Systems	differentiate between the various types of distributed computing models.	Categorizes distributed systems into three main types: Distributed Computing Systems (Cluster, Grid), Distributed Information Systems (Transaction processing, Enterprise systems), and Distributed Pervasive Systems (IoT, Smart homes).

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 8: Distributed System Architectures (Part I)	Understand centralized architectural models and their physical tiered layouts.	Focuses on Client-Server architecture. It breaks down Two-tier, Three-tier, and Multi-tier (N-tier) architectures, discussing their advantages, limitations, and examples like banking and university systems.

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 9: Distributed System Architectures (Part II)	Analyze decentralized architectures and modern service-oriented approaches.	Explores Peer-to-Peer (P2P) architectures (Centralized, Decentralized, Hybrid), Service-Oriented Architecture (SOA), and Microservices. Uses BitTorrent, Skype, and Netflix as real-world case studies.

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 10: Communication in Distributed Systems	Explain the mechanisms and paradigms used for interprocess communication (IPC).	Covers IPC methods (Message Passing, Shared Memory), Sockets and Ports, Synchronous vs. Asynchronous communication, Remote Procedure Call (RPC), Remote Method Invocation (RMI), and the role of Middleware.

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 11: Synchronization in Distributed Systems	Solve the problem of event ordering and time coordination without a global clock.	Explains physical clocks (UTC, Clock Drift, Skew) and why they are unreliable. Introduces Logical Clocks (Lamport and Vector clocks) and the "Happened-Before" relationship to maintain causality.

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 12: Distributed Concurrency Control	Manage concurrent access to shared data and prevent incorrect result.	Introduces ACID properties, interleaving, and concurrency problems (Lost Update, Dirty Read, Inconsistent Retrieval). Covers control mechanisms like Mutual Exclusion, Locks, Two-Phase Locking (2PL), and Deadlocks.

ACID properties
A = Atomicity (All or Nothing), C = Consistency (Data Stays Correct), I = Isolation (No Interference), D = Durability (Permanent Changes)

Course Objectives

What you will achieve in this course

lecture	Objective	Description
Lecture 13: Distributed Algorithms	Understand how distributed nodes coordinate, agree, and elect leaders without a central master.	Covers algorithm characteristics, fault tolerance, and failure types. Details Election Algorithms (Bully and Ring) and introduces modern consensus protocols like Paxos, Raft, and Gossip.

Consensus = All distributed nodes agree on the same value.
Paxos = Reliable but complex consensus algorithm.
Raft = Easier-to-understand consensus algorithm with a leader.

Course Objectives

Tools Needed for Your Distributed Systems Labs

Based on the Java labs you've been discussing (Sockets, Threads, Lamport Clocks, Vector Clocks, RPC, Leader Election, Distributed Locks, etc.), the minimum setup is:

Tool	Required?
JDK 17+	✅ Yes
IntelliJ IDEA / NetBeans	✅ Yes
Command Prompt / Terminal	✅ Yes
Localhost Network	✅ Yes
MySQL	✅ For database labs
MySQL Workbench	✅ Helpful
Docker	(optional)Simulates multiple machines on a single PC.

Course Objectives

Tools Needed for Your Distributed Systems Labs

With these tools, students can implement nearly all basic distributed systems experiments:

- Client-Server Communication
- TCP Socket Programming
- Message Passing
- RPC Simulation
- RMI
- Lamport Clocks
- Vector Clocks
- Leader Election (Bully/Ring)
- Mutual Exclusion
- Distributed Lock Manager
- Concurrency Control
- Load Balancing
- Microservices Simulation
- Failure Detection

References & Resources

What you need for this course

Primary Textbook

- Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012).
- Distributed Systems: Concepts and Design (5th Ed.).
- Addison-Wesley.

Secondary Textbook

- Tanenbaum, A. S., & Van Steen, M. (2007).
- Distributed Systems: Principles and Paradigms (2nd Ed.).
- Pearson Prentice Hall.

Assessment Methods

How your grade will be calculated

Component	Weight	Description
Lab Assignments	10%	Java programming (Sockets, RMI, Threads) to implement DS concepts.
Midterm Exam	20%	Covers Lectures 1-7 (Architecture, IPC, Synchronization, Clocks).
Final Exam	60%	Covers all Lectures from 1-7 and from 8-13 (Transactions, Concurrency, Algorithms, Consensus).
Project / Participation	10%	Class activities, discussions, and distributed system design project.